Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 3

Experiment Question

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Aim

To build a decision tree using the ID3 algorithm on the PlayTennis dataset, and to use the tree to classify a new, unseen sample.

Concept

A **decision tree** classifies an example by sorting it down the tree from the root to a leaf. Each internal node tests one attribute, each branch is a value of that attribute, and each leaf gives a class label.

ID3 (Iterative Dichotomiser 3) builds the tree top-down with a greedy strategy. At each node it chooses the attribute with the highest **information gain**, that is, the attribute that best reduces the uncertainty (**entropy**) of the target class. The data is then split on that attribute, and the same process is repeated for each branch with the remaining attributes. A branch becomes a leaf when all its examples have the same class, or when no attributes are left.

Dataset Description

The PlayTennis dataset is stored in a CSV file named `playtennis.csv`. It has 14 examples, 4 input attributes (Outlook, Temperature, Humidity, Wind) and 1 target attribute (PlayTennis = Yes/No). There are 9 Yes and 5 No examples.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Contents of playtennis.csv (the Day column is not used):

```
Outlook,Temperature,Humidity,Wind,PlayTennis
Sunny,Hot,High,Weak,No
Sunny,Hot,High,Strong,No
Overcast,Hot,High,Weak,Yes
Rain,Mild,High,Weak,Yes
Rain,Cool,Normal,Weak,Yes
Rain,Cool,Normal,Strong,No
Overcast,Cool,Normal,Strong,Yes
Sunny,Mild,High,Weak,No
Sunny,Cool,Normal,Weak,Yes
Rain,Mild,Normal,Weak,Yes
Sunny,Mild,Normal,Strong,Yes
Overcast,Mild,High,Strong,Yes
Overcast,Hot,Normal,Weak,Yes
Rain,Mild,High,Strong,No
```

Algorithm

```
ID3(Examples, Target_attribute, Attributes)
1. Create a Root node for the tree.
2. If all Examples have the same class, return a leaf node with that class.
3. If Attributes is empty, return a leaf node with the most common class in Examples.
4. Otherwise:
    a. A = the attribute in Attributes with the highest Information Gain
    b. Set the decision attribute of Root to A
    c. For each value v of A:
        - Add a new branch below Root for A = v
        - Examples_v = the examples in which A = v
        - Attach the subtree  ID3(Examples_v, Target_attribute, Attributes - {A})
5. Return Root
```

Step-by-step Procedure

1. Read the training data from `playtennis.csv`.
2. Calculate the entropy of the whole dataset with respect to the target class.
3. For every attribute, calculate the information gain.
4. Select the attribute with the highest gain as the decision node (the root is Outlook).
5. Split the dataset on each value of that attribute.
6. For each subset: if all its examples have the same class, make it a leaf; otherwise repeat steps 3 to 5 using the remaining attributes.
7. Print the final decision tree.
8. Classify a new sample by following the branches that match its attribute values until a leaf is reached.

Formulae

Entropy of a set S with classes i = 1 to c, where p_i is the proportion of examples in class i:

```
Entropy(S) = -\sum_{i=1}^{c} p_i \log_2 p_i
```

Information gain of attribute A, where S_v is the subset of S for which A has value v:

```
Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \, Entropy(S_v)
```

Calculation for the root node:

- Entropy(S) with 9 Yes and 5 No = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.9403$
- Gain(S, Outlook) = 0.2467
- Gain(S, Temperature) = 0.0292
- Gain(S, Humidity) = 0.1518
- Gain(S, Wind) = 0.0481

Outlook has the highest gain, so it becomes the root node.

Python Program

```
import csv
import math
from collections import Counter

# Step 1: Load the dataset
with open('playtennis.csv') as f:
    data = list(csv.DictReader(f))

columns = list(data[0].keys())
attributes = columns[:-1]
target = columns[-1]

# Entropy of a set of examples
def entropy(rows):
```

```
    counts = Counter(r[target] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

# Information gain of an attribute
def info_gain(rows, attr):
    total = len(rows)
    remainder = 0
    for value in sorted(set(r[attr] for r in rows)):
        subset = [r for r in rows if r[attr] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

# ID3 algorithm (returns a nested dictionary)
def id3(rows, attrs):
    labels = set(r[target] for r in rows)
    if len(labels) == 1:                    # pure node -> leaf
        return labels.pop()
    if not attrs:                           # no attributes left -> majority class
        return Counter(r[target] for r in rows).most_common(1)[0][0]

    gains = {a: round(info_gain(rows, a), 4) for a in attrs}
    best = max(gains, key=gains.get)
    print(f'Examples: {len(rows)} | Gains: {gains} | Best: {best}')

    tree = {best: {}}
    remaining = [a for a in attrs if a != best]
    for value in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == value]
        tree[best][value] = id3(subset, remaining)
    return tree

# Display the tree
def print_tree(tree, indent=''):
    if not isinstance(tree, dict):
        print(indent + '->', tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f'{indent}{attr} = {value}')
        print_tree(subtree, indent + '    ')

# Classify a new sample
def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

# Build the tree
print('Entropy of the dataset:', round(entropy(data), 4))
```

```
print()
tree = id3(data, attributes)

print('\nDecision Tree:')
print_tree(tree)

# Classify a new sample
new_sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
print('\nNew sample:', new_sample)
print('Predicted class (PlayTennis):', classify(tree, new_sample))
```

Output

```
Entropy of the dataset: 0.9403

Examples: 14 | Gains: {'Outlook': 0.2467, 'Temperature': 0.0292, 'Humidity': 0.1518, 'Wind': 0.0481} |
Best: Outlook
Examples: 5 | Gains: {'Temperature': 0.02, 'Humidity': 0.02, 'Wind': 0.971} | Best: Wind
Examples: 5 | Gains: {'Temperature': 0.571, 'Humidity': 0.971, 'Wind': 0.02} | Best: Humidity

Decision Tree:
Outlook = Overcast
   -> Yes
Outlook = Rain
   Wind = Strong
      -> No
   Wind = Weak
      -> Yes
Outlook = Sunny
   Humidity = High
      -> No
   Humidity = Normal
      -> Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Predicted class (PlayTennis): No
```

Result / Conclusion

The ID3 algorithm was implemented in Python and a decision tree was built on the PlayTennis dataset. Outlook was selected as the root because it has the highest information gain (0.2467). Under Sunny, the tree tests Humidity; under Overcast, the answer is always Yes; under Rain, the tree tests Wind.

The new sample (Sunny, Cool, High, Strong) was classified as **PlayTennis = No**. Hence the working of the ID3 decision tree algorithm was successfully demonstrated.